中图分类号：TN492

学号：ZY2402113

北京航空航天大學

硕士中期检查报告

基于RISC-V的
软硬件协同及编译

作者姓名 蒋一平

学科专业 电子信息

指导教师 李洪革教授，李丽副教授

培养院系 电子信息工程学院

2026年10月

目录

1.开题报告的论文研究方案 1
1.1 研究背景与意义 1
1.2 主要研究内容 1
1.3 关键技术与预期新意 2
2.开题报告的论文工作计划 3
3.已研究的内容和实现技术及关键技术的突破 4
3.1 Linux系统移植与驱动开发 4
3.1.1 系统总体方案 4
3.1.2 轻量级引导程序 4
3.1.3 内核裁剪与根文件系统 5
3.1.4 设备树与外设驱动 6
3.1.5 系统集成与运行验证 7
3.2 AI编译器开发 8
3.2.1 总体设计与部署方式 8
3.2.2 模型解析与参数整理 10
3.2.3 内存规划设计 11
3.2.4 可执行文件生成 13
3.2.5 系统验证与阶段成果 14
4.已研究工作总结 15
4.1 Linux系统相关 15
4.2 AI编译器相关 16
5.后续研究的内容和实现技术 16
5.1 内存规划优化 16
5.2 算子融合 17
5.3 实验验证与工作安排 18
6.已发表的论文和取得的成果 20
7.参考文献 21

1.开题报告的论文研究方案

1.1 研究背景与意义

近年来，随着嵌入式系统与人工智能应用的发展，端侧设备对计算性能、资源利用效率和系统可定制性的要求不断提高。RISC-V指令集具有开放、模块化和可扩展等特点，为构建自主可控的处理器及片上系统提供了基础。基于FPGA构建集成RISC-V处理器、显示模块和NPU的SoC，能够为软硬件协同设计提供可重构的实验平台。然而，硬件功能的实现还需要操作系统、设备驱动和编译工具的配套支持。本课题以“基于RISC-V的软硬件协同及编译”为研究主题，围绕Linux系统移植及驱动开发、面向NPU的AI编译器构建与优化两个方面开展研究，以完善自研SoC的软件支持并提升其面向实际应用的运行能力。

在操作系统方面，Linux具有开源、模块化和可移植性较强等特点，能够为应用程序提供进程管理、存储管理和设备访问等基础服务，是完善RISC-V软件环境的重要组成部分。针对FPGA平台资源有限、处理器运行频率受约束以及自研外设缺少配套驱动等问题，需要结合目标SoC设计轻量级引导程序，完成Linux内核、设备树和根文件系统的适配与裁剪，并为图像显示模块、NPU等外设开发相应驱动。通过对启动流程、系统组成与外设访问方式进行协同设计，可以在满足应用功能的基础上控制系统体积和启动开销，为图形交互与异构计算应用提供统一、可靠的软件运行环境。

在AI编译器方面，神经网络通常以计算图和张量运算描述，而实验室自研NPU采用具有特定编码、数据格式和地址要求的宏指令，两者之间存在模型语义、物理布局和执行控制等方面的差异。AI编译器需要通过计算图解析、算子映射、静态存储规划与指令生成，将模型转换为目标NPU能够执行的程序。已有研究表明，图级优化与算子级映射共同影响模型在目标平台上的执行效率[1]。因此，面向自研NPU构建专用离线编译器，不仅能够建立模型到硬件执行的自动转换流程，还能够结合宏指令粒度、通道对齐及张量依赖等约束，研究执行调度与存储布局的优化方法，为发挥专用加速器的计算能力提供编译支持。

1.2 主要研究内容

本课题的研究内容主要包括以下两个方面。

（1）RISC-VSoC的Linux系统移植及驱动开发。以FPGA实现的RISC-VSoC为基础，研究系统启动、内核适配、根文件系统构建以及外设访问所需的软件支持，建立处理器与片上外设之间的基本软件接口。该部分工作关注操作系统与硬件平台之间的适配关系，为后续系统应用提供支撑。

（2）面向NPU的AI编译器构建与优化。围绕神经网络等AI模型从模型描述到NPU执行的转换过程，研究模型导入、中间表示、算子合法化、张量布局、参数组织、存储地址规划以及指令生成等关键问题。在完成离线编译与板端部署的基础上，进一步分析宏指令执行方式和多分支网络的数据依赖，探索降低中间张量存储占用及冗余数据搬运的方法。

编译部分采用部署前完成主要决策的技术路线。对于结构和输入形状确定的模型，在PC侧生成固定执行计划、指令序列及参数数据，设备侧负责加载编译产物、配置执行环境、启动NPU并获取结果。TVM等研究说明了将模型级优化与硬件相关代码生成相结合的可行性[2]；本课题根据自研NPU的指令和存储特征，构建范围明确、能够直接服务于原型系统的编译流程。

1.3 关键技术与预期新意

Linux系统方面，关键技术包括面向自研RISC-V SoC的启动环境构建、精简SBI服务实现、Linux内核与根文件系统裁剪，以及设备树适配和定制外设驱动开发。结合开题报告中的研究方案，预期新意在于围绕FPGA平台的资源与频率约束，对引导程序、操作系统配置和外设访问进行协同设计：自主开发仅包含必要硬件初始化、内核加载及启动支持的轻量级引导程序，并结合目标应用裁剪内核与根文件系统，在功能完整性、系统体积和启动效率之间进行权衡；同时，针对图像显示模块建立相应驱动接口，结合硬件条件组织中断响应和数据传输，支撑处理器、显示模块的协同运行，形成面向本SoC的定制化Linux软件支持方案。

AI编译器方面，关键技术与预期新意主要包括两个方面。一是面向实验室自研、采用宏指令体系的NPU构建专用离线编译器，针对单条指令执行较大粒度张量操作的特点，研究计算图到宏指令执行序列的映射方法，将算子分解、通道分组、参数组织及静态地址规划相结合，在PC侧生成与目标硬件匹配的指令和参数二进制文件，建立从模型描述到RISC-VSoC中NPU执行的编译路径。二是在已有编译流程基础上，研究面向宏指令及物理布局约束的执行调度与存储联合优化，将张量生

命周期、分支依赖、通道对齐和拼接别名关系纳入统一分析，联合选择合法执行顺序与缓冲区复用方案，降低中间张量的峰值存储需求，并减少具备消除条件的数据搬运。前者体现面向自研硬件的专用编译适配，后者作为后续重点研究的优化方法，通过同一硬件平台上的对照与消融实验评价其有效性。

2.开题报告的论文工作计划

开题阶段的总体工作安排按照系统构建、软件适配、编译优化与论文整理依次展开。结合目前已经具备的NPU编译和FPGA验证条件，在保持系统软件与编译两个研究方向的基础上，将编译部分的工作进一步细化为流程完善、约束分析、优化算法实现和板端实验。原计划与当前阶段安排如表1所示。

表1 论文研究工作计划

时间安排	计划内容	阶段说明
2025.12—2026.02	构建RISC-VSoC与FPGA验证环境，集成相关外设	原计划中的硬件平台阶段
2026.03—2026.04	研究启动加载与基础软件适配	系统软件方向
2026.05—2026.07	Linux内核与根文件系统移植	已完成，详见3.1节
2026.08—2026.09	外设驱动开发与系统联调	已完成，详见3.1节
2026.10—2026.12	面向NPU的AI编译与优化研究	已有全流程基线，继续开展方法与实验
2027.01—2027.04	补充实验、撰写与修改学位论文	优化验证与论文整理交叉进行
2027.05	论文答辩	按学院实际安排执行

3.已研究的内容和实现技术及关键技术的突破

3.1 Linux系统移植与驱动开发

3.1.1 系统总体方案

按照开题报告确定的研究方案，本阶段已完成自研RISC-VSoC上的Linux系统移植及驱动开发，建立了从处理器复位、引导程序执行、内核启动到用户态应用运行的完整软件环境。系统以FPGA实现的RISC-VSoC为硬件基础，围绕处理器、存储控制器、中断控制器、UART、显示与触摸模块以及NPU等设备完成软件适配，使操作系统能够管理硬件资源，并为应用程序提供统一的设备访问接口。

Linux软件系统由引导程序、内核、设备树和根文件系统共同构成，各部分的组成及构建关系如图1所示。其中，引导程序负责准备内核启动环境；Linux内核提供进程管理、内存管理和设备管理等基础功能；设备树描述SoC的硬件资源；根文件系统提供命令行工具、运行库及应用程序。针对FPGA平台的资源和运行频率约束，系统采用定制引导程序、内核功能裁剪和精简根文件系统相结合的实现方式，在满足目标应用需求的基础上控制软件规模。

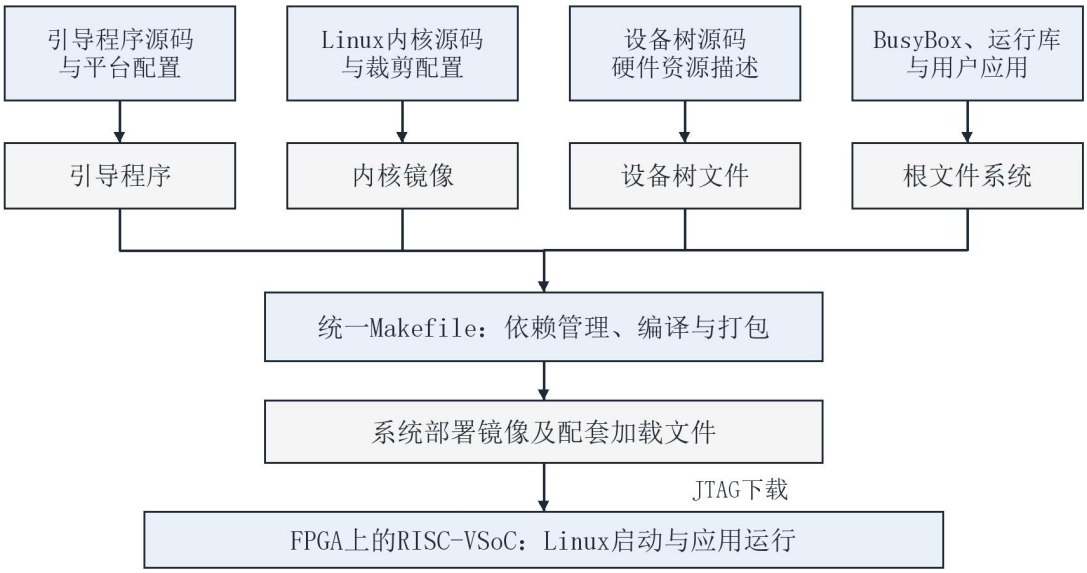


图1 Linux系统组成与构建流程

3.1.2 轻量级引导程序

引导程序是处理器复位后执行的基础软件，其主要任务是初始化关键硬件、建立必要的异常处理与固件服务，并将控制权交给Linux内核。针对本SoC的固定硬

件组成，本阶段已完成轻量级定制引导程序的设计与实现，围绕硬件初始化、内存设置、内核加载及启动参数准备组织执行流程，减少与目标平台无关的功能配置。

在启动早期，引导程序完成运行环境设置，并配置异常入口及必要的上下文保存、恢复机制；随后初始化UART，使启动过程能够通过串口输出调试信息，同时完成中断控制器、定时器等关键外设的基础设置。异常处理部分按事件类型组织分发和处理流程，为启动阶段的故障定位以及后续固件服务调用提供支持。图2给出了引导程序的执行过程。

针对Linux内核与底层硬件之间的调用需求，已实现满足本系统启动和运行需要的最小化SBI服务，包括所需的控制台与定时器支持。内核通过约定接口请求底层服务，由固件侧完成相应处理。在启动交接阶段，引导程序将内核映像置于预定内存区域，准备设备树地址等启动信息，设置必要的异常委托与处理器运行状态，再转入Linux内核入口。通过上述实现，完成了从复位环境到操作系统运行环境的衔接。

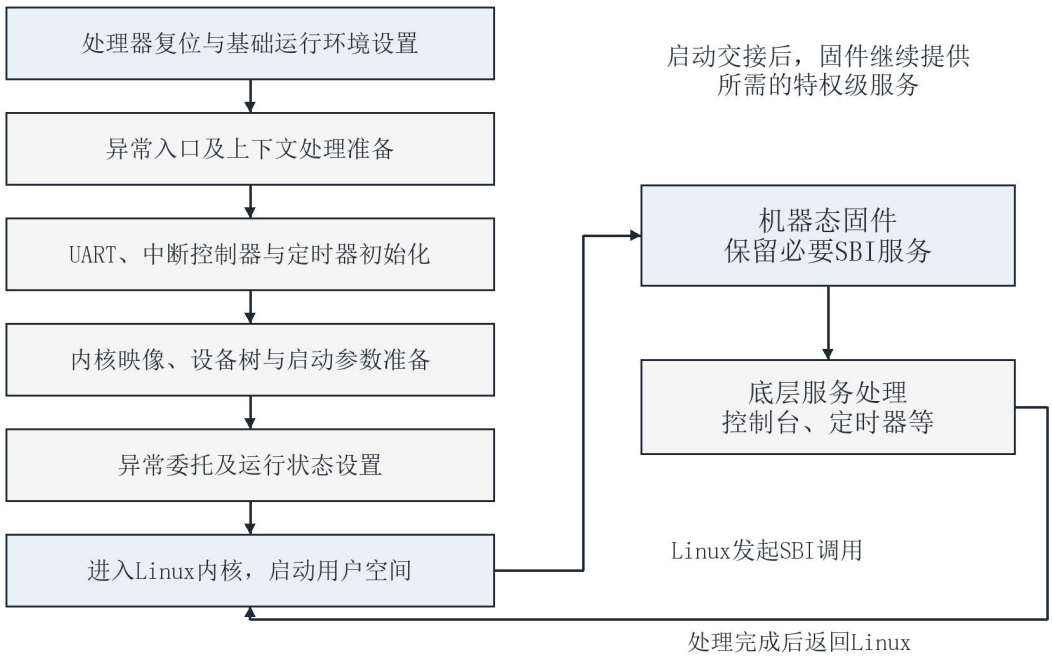


图2 RISC-V引导程序执行流程

3.1.3 内核裁剪与根文件系统

Linux内核移植围绕目标处理器能力、内存布局和外设组成进行配置。本阶段已完成RISC-V体系结构相关配置和平台资源适配，使内核能够识别目标SoC的处理器、内存及必要外设，并完成系统初始化。针对FPGA平台资源有限的特点，结合

menuconfig等配置工具对内核功能进行裁剪，保留进程管理、虚拟内存管理、根文件系统支持及目标应用所需的驱动，移除未使用的设备、文件系统和其他可选功能。

裁剪过程中，以系统能够完整启动和目标应用能够正常运行为基本条件，逐项检查配置依赖，避免因删除支撑模块导致驱动无法初始化或用户程序无法运行。调试阶段保留必要的串口日志及定位手段，在功能稳定后精简非必要的调试和分析配置。通过将内核配置与本SoC的硬件组成相对应，已形成适用于目标平台的定制内核镜像，为外设驱动和上层应用提供运行基础。

根文件系统采用开题报告中确定的BusyBox结合手动构建方案。已完成BusyBox的交叉编译与功能配置，建立基本目录结构，并补充系统运行所需的库文件、设备节点、初始化脚本及测试应用。启动脚本负责组织用户空间初始化，准备必要的运行环境并进入命令行或应用程序，使内核启动后能够继续完成用户态功能。

通过内核与根文件系统的配合，系统已能够从内核初始化进入用户空间，执行基本命令并运行外设测试程序。裁剪后的内核配置、BusyBox配置及根文件系统组成共同构成当前软件基线，便于在后续功能扩展时定位新增模块带来的空间和启动开销。镜像体积与启动时间的具体结果将在最终实验材料中按统一统计口径补充，分别说明内核映像、根文件系统和完整部署镜像的大小，以及启动计时的起止位置。

3.1.4 设备树与外设驱动

设备树用于向Linux描述SoC的硬件组成，是内核与自研外设建立联系的重要依据。本阶段已完成处理器、内存、中断控制器、定时器及各外设的设备树适配，描述相应寄存器地址范围、中断资源和设备间的连接关系，并使其与硬件地址映射保持一致。驱动根据设备描述获取所需资源，完成设备识别、初始化和接口注册，将底层寄存器操作封装为应用程序能够使用的访问方式。

对于UART等通用外设，采用内核现有驱动并完成平台配置；对于需要定制适配的中断控制器、显示与触摸模块以及NPU，则依据对应硬件接口完成驱动开发。驱动实现围绕资源获取、寄存器读写、运行状态管理和中断响应展开，并完成与内

核构建配置及设备树的集成。通过分别检查设备匹配、资源地址和中断处理，建立了从硬件事件到内核响应的完整路径。

图像显示部分已完成显示模块及相关DMA传输的驱动适配，按照硬件要求配置显示数据缓冲区、传输地址和控制状态，使应用程序生成的图像数据能够送至LCD显示。触摸输入部分完成输入信息读取与上报，并与显示应用结合实现基本人机交互。该过程需要保持图像格式、缓冲区布局和硬件读取方式一致，同时协调数据传输、状态更新和事件处理，确保显示与输入功能能够配合运行。

NPU驱动完成了加速器资源管理与执行控制接口的适配，为上层程序提供参数和指令数据准备、地址配置、计算启动及状态获取等支持。通过驱动将NPU控制操作纳入Linux设备访问流程，应用程序能够在操作系统环境中组织处理器与加速器的协同工作。显示、触摸及NPU等外设的访问关系如图3所示，各驱动分别处理对应硬件细节，上层应用按功能调用相应接口。

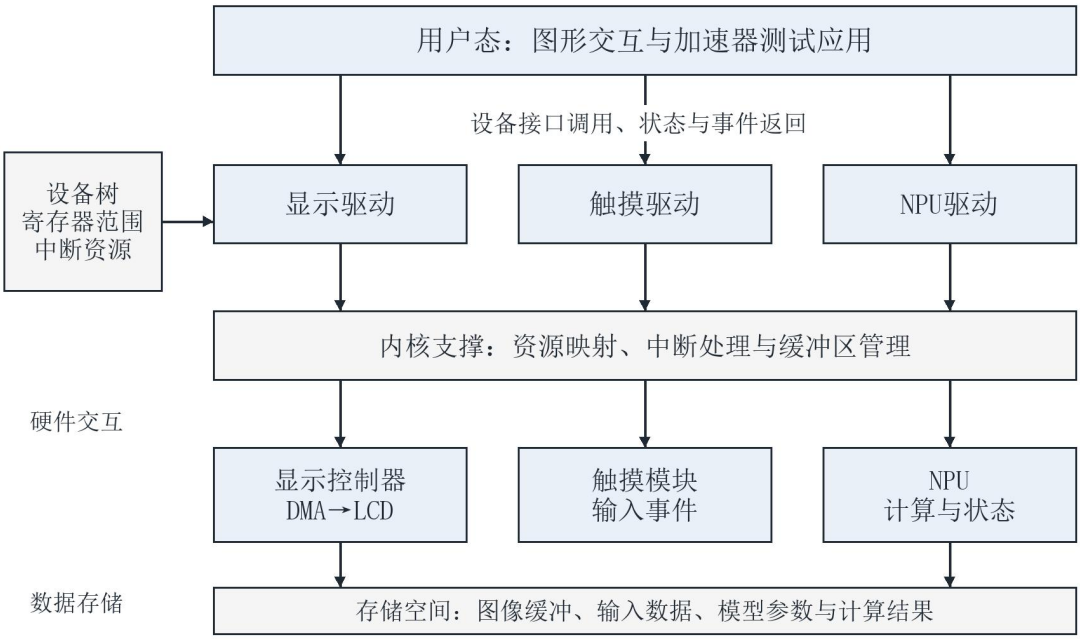


图3 Linux设备驱动与应用协同关系

3.1.5 系统集成与运行验证

在各模块完成后，已通过统一Makefile组织引导程序、Linux内核、设备树及根文件系统的编译与打包，形成可用于目标SoC的系统镜像。构建过程统一管理各部分的配置与产物依赖，将内核、设备描述和用户态环境组合为完整部署内容，并通过JTAG下载至FPGA平台进行系统联调。统一构建流程使配置修改能够传递到相应产物，便于重复生成和验证同一软件版本。

系统验证按照启动过程和应用功能逐步展开。首先通过串口观察引导程序初始化、内核启动、设备识别和根文件系统进入情况，确认从处理器复位到用户空间的运行链路；随后运行基本命令及外设测试程序，验证UART、显示、触摸和NPU等设备的访问功能；最后通过应用程序组织外设协同工作，并在LCD上完成显示与交互演示。上述移植、驱动开发和整体联调工作均已完成，实现了开题报告所确定的Linux系统软件功能。

图4待插入：插入真实板端运行材料，建议组成三幅子图：（a）UART启动日志，保留引导程序输出、Linux内核初始化和进入用户空间的关键内容；（b）LCD显示及触摸交互实物照片；（c）Linux下NPU驱动测试或应用调用的运行记录，展示任务启动与完成结果。注明板卡型号、CPU工作频率、内核版本及测试日期；镜像大小和启动耗时由对应构建产物与日志记录填写。

图4 Linux启动及外设应用运行结果（图片待补充）

本阶段形成了轻量级引导程序、内核配置与镜像、设备树、定制外设驱动、BusyBox根文件系统以及统一构建脚本等软件成果。系统已具备基本命令行、图形交互和加速器访问能力，解决了自研硬件从裸机控制到操作系统管理的适配问题，并为后续AI编译产物部署、应用联调和性能测量提供了配套运行环境。

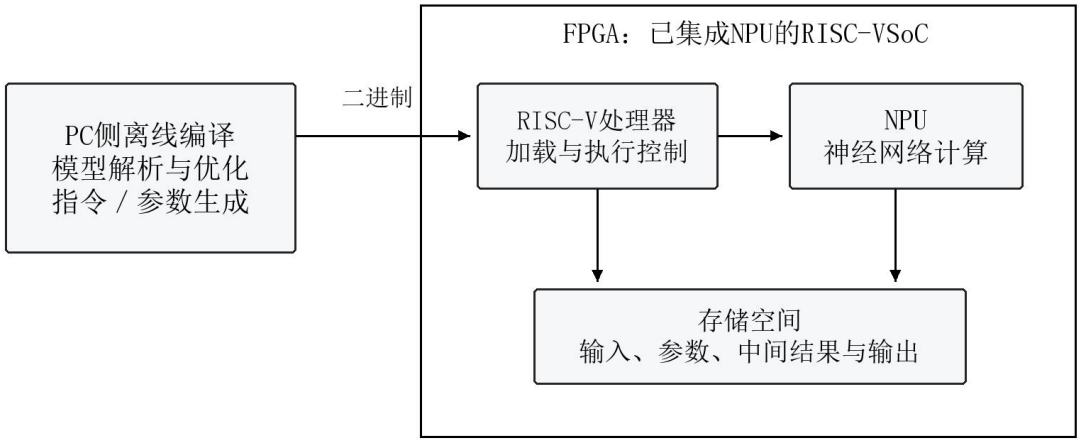
3.2 AI编译器开发

3.2.1 总体设计与部署方式

本课题围绕实验室自研NPU构建离线AI编译器，将神经网络模型转换为目标硬件能够直接执行的二进制文件。目前已完成模型解析、参数整理、内存规划、指令生成及文件导出等主要模块，并在集成NPU的RISC-VSoC上完成FPGA验证，能够运行编译器生成的YOLO网络二进制文件并正确进行图像识别。编译器的设计目标是将原先需要人工完成的模型信息提取、数据排列、地址计算和指令配置组织为统一流程，为自研NPU的模型部署与后续优化提供软件基础。

编译系统采用PC侧编译、SoC侧加载执行的部署方式，如图5所示。在PC侧，编译器依据模型结构和目标硬件约束，预先确定运算顺序、参数布局及中间结果地址，生成完整的NPU指令流。部署时，将指令、参数和输入数据装载到SoC的指定存储区域，由RISC-V处理器完成必要的地址配置与启动控制，随后NPU按指令流

读取数据、执行计算并写回结果。端侧不需要重新解析模型计算图，加载与控制程序主要负责连接编译产物和硬件执行接口。



系统结构示意；实际总线与加载接口以平台实现为准

图5 离线AI编译器与RISC-V异构SoC的关系

编译器总体流程如图6所示，主要由模型前端、内存规划、算子指令生成和文件导出等部分组成。前端将模型结构转换为统一的中间表示（IR），并提取权重、偏置等参数；内存规划模块结合硬件约束确定计算过程及数据地址；后端据此生成汇编指令，经汇编编码后形成机器码，最终输出部署所需文件。各阶段通过模型结构文件和内存规划文件传递信息，使模型解析与硬件指令生成保持相对独立。

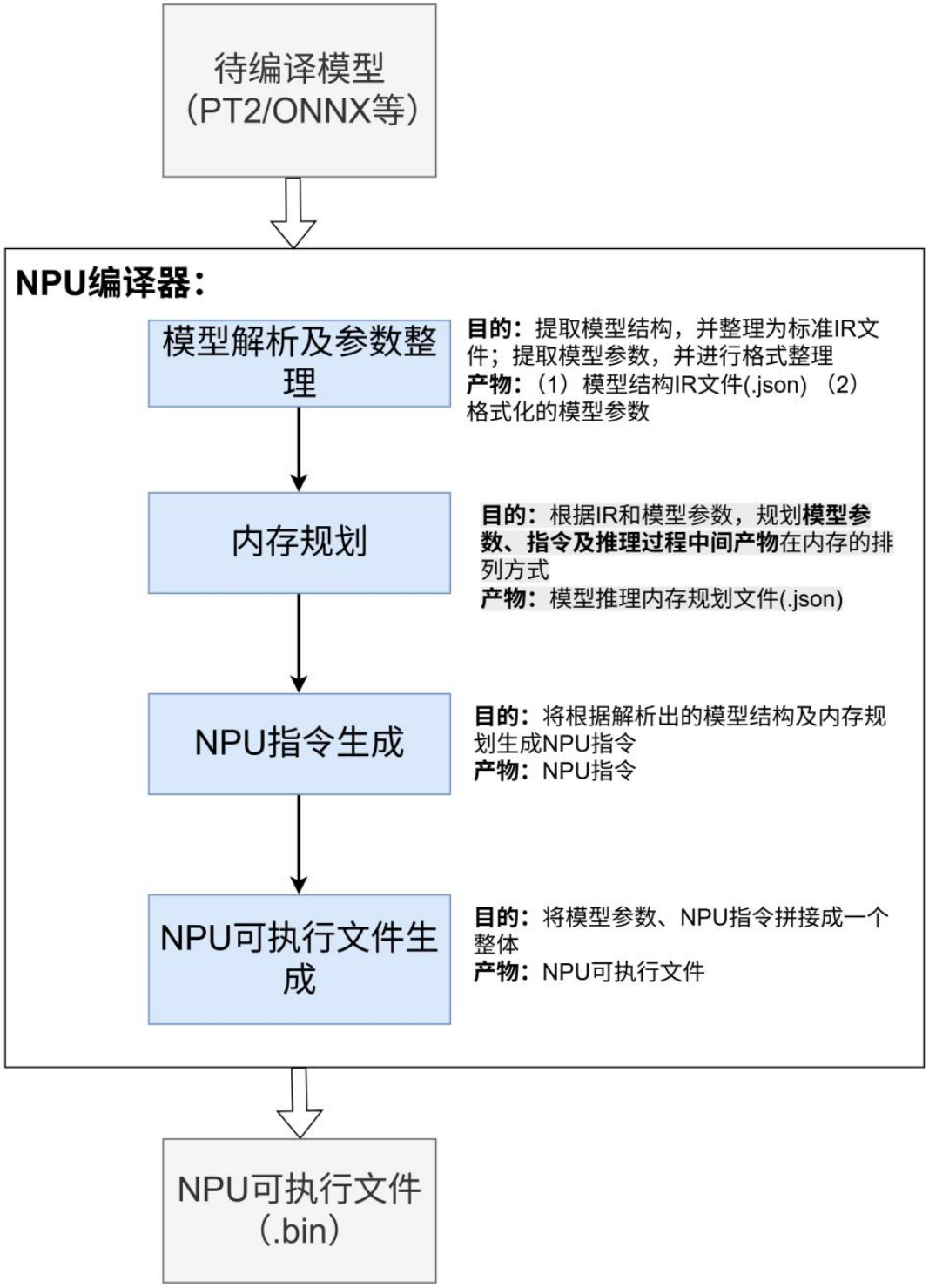


图6 NPU编译器总体流程

3.2.2 模型解析与参数整理

模型解析的主要任务是从原始模型中提取计算结构与参数，并转换为后续模块能够统一处理的形式。目前前端以PyTorch导出的PT2模型为输入，读取计算节点、张量形状及参数映射关系，将算子类别、输入输出连接、卷积核大小、步长和填充等信息写入图中间表示，同时保存模型权重与偏置。统一的IR使后端能够依据

模型语义完成处理，而不必在每个编译阶段重复读取原始模型。该分层方式与通过中间表示逐级完成模型到目标代码转换的思路一致[3]。

以连续的卷积和ReLU运算为例，前端首先记录两个节点的运算属性及数据依赖，明确ReLU的输入来自卷积输出；随后提取卷积所需的权重和偏置，供参数生成与地址规划使用。现有前端已建立卷积、激活、池化、最近邻上采样、填充、通道拼接和全连接等操作到内部表示的映射，具体可执行的参数范围由后续硬件适配规则进一步约束。若需要支持其他模型格式，可增加相应解析入口，将其转换为相同的IR，并完成算子语义和数值行为的校验后复用后端流程。

模型参数还需按照NPU的数据读取方式进行整理。框架导出的权重排列与硬件使用的通道分组、像素交织顺序可能不同，因此参数生成模块需要完成权重重排、必要的通道补齐及偏置组织。对于采用4通道对齐的情况，编译器以通道组为单位安排存储，将不足一组的部分补齐，使指令所描述的通道数量与实际参数空间相匹配。图7给出了逻辑通道与物理存储之间的关系。

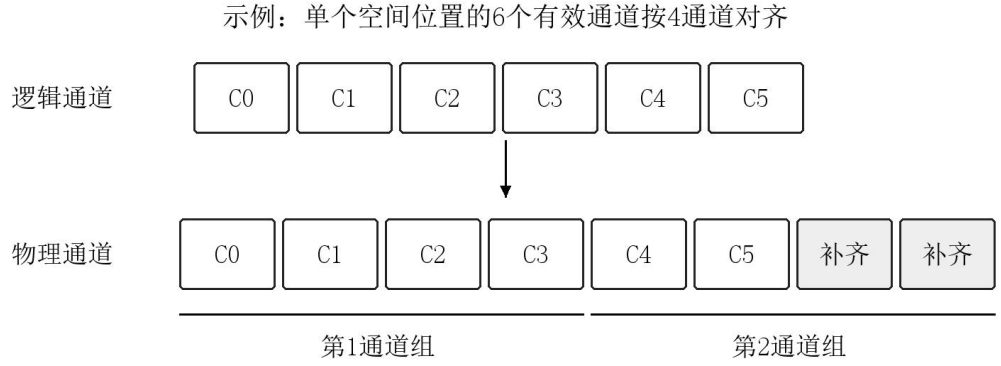


图7 通道对齐对物理存储空间的影响示意

参数整理与执行计划共同决定硬件实际读取的数据。例如，输出通道较多的卷积会根据目标硬件能力拆分为多个计算分组，每个分组需要对应的权重范围和输出区域。编译器依据分组结果生成参数数据及偏移信息，保证后续指令能够访问正确的参数。对于具有偏置和不具有偏置的操作，也分别组织相应内容，避免参数长度或排列方式与指令配置不一致。

3.2.3 内存规划设计

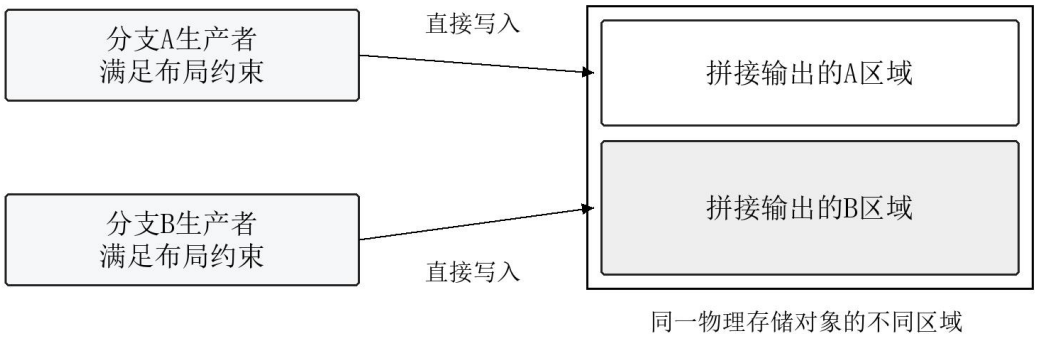
内存规划是连接模型描述与指令生成的关键环节。该阶段以统一IR和硬件配置为依据，确定输入数据、模型参数、中间张量、模型输出及指令所需的存储空间，并形成memory_plan.json等规划文件。规划结果不仅记录各区域的起始地址和大

小，还包含算子分组、参数偏移及输出位置等执行信息，供参数生成、指令生成与文件导出共同使用。通过在编译阶段完成这些工作，可以减少人工计算地址的工作量，并保持各阶段对同一数据的解释一致。

仍以卷积后接ReLU为例，规划器首先根据输入形状、卷积属性和通道对齐规则计算卷积输出的物理大小，为其分配存储区域；随后将该输出地址作为ReLU的输入地址，并为激活结果安排输出位置。与只按逻辑张量元素数量估计空间相比，实际地址计算还需要考虑对齐填充、分组方式和硬件寻址规则。规划完成后，编译器检查相关地址区间的边界及重叠情况，后端据此生成一致的寄存器配置。

针对模型运算规模与NPU单次计算能力不一致的情况，内存规划同时组织相应的执行步骤。例如，卷积输出通道可按硬件支持的粒度进行分组，逻辑上的一次卷积由多次计算完成，各组分别使用对应的参数和输出区域。对于部分需要组合实现的运算，则依据目标指令能力安排相邻操作及数据连接。这一过程将计算图中的逻辑节点转换为可以实际执行的操作序列，使后续指令生成集中于具体字段的配置和编码。

对于满足条件的通道拼接，当前编译器采用预先安排生产者输出位置的方法，使各分支直接写入最终拼接张量的对应区域，如图8所示。当拼接方向、空间尺寸及通道对齐满足目标布局要求时，后续算子可以直接读取该区域，从而省去单独的拼接数据拷贝。这一处理将网络中的连接关系与物理存储布局结合起来，减少了部分不必要的数据搬运。



无额外拼接拷贝；复用时必须共同检查别名与消费者

图8 满足布局条件时的通道拼接免拷贝示意

目前中间结果区域仍以顺序分配为主，尚未实现面向全图生命周期的通用缓冲区复用。随着网络深度和分支数量增加，已完成消费的中间张量仍可能占用原有空间。同时，拼接预布局会使多个逻辑张量共享同一物理存储对象的不同区域，复用

时需要统一处理其别名和使用关系。因此，现有规划文件也为后续开展生命周期分析、分支调度与存储联合优化提供了基础。

3.2.4 可执行文件生成

在模型执行顺序、数据布局和存储地址确定后，算子编译模块读取内存规划文件，为各执行节点生成对应的汇编片段。目标NPU采用宏指令形式，通常先通过配置指令设置输入地址、输出地址、参数地址和算子属性，再由计算指令触发相应硬件单元。编译器因此需要在指令生成前完成尺寸、通道分组和地址等静态计算，使硬件能够依据预先生成的配置执行运算。

以卷积运算为例，后端先设置输入、输出和权重地址所在的通用寄存器，再配置卷积核大小、通道数量及偏置等相关字段，最后生成CONV指令启动计算；后续ReLU节点则引用卷积结果地址并生成相应配置与计算指令。图9给出了卷积与ReLU的汇编示例，展示了地址配置、算子属性配置和计算启动之间的关系。图中地址用于说明该示例的编译结果，实际模型的地址由对应内存规划决定。

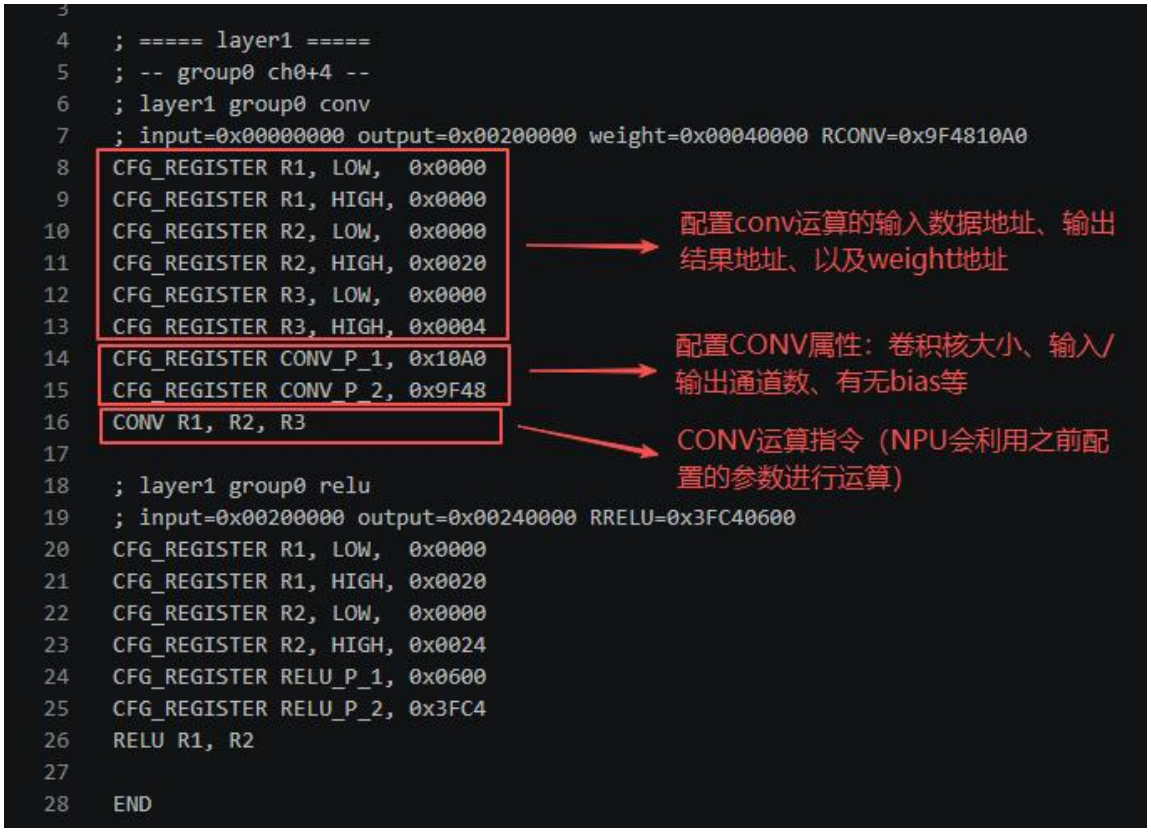


图9 卷积与ReLU的汇编指令示例

汇编器按照目标指令编码规则，将上述汇编序列转换为32位机器码。构建脚本进一步组织参数与指令导出：对于BRAM仿真和初始化场景，可生成按预定布局组

织的COE等文件；对于DDR部署场景，则分别输出npu_instr.bin与npu_params.bin，供系统按指定基地址加载。指令编码、参数排列和装载位置均与内存规划保持对应，构成从模型到硬件执行文件的完整转换过程。

分层的后端结构也为增加硬件算子提供了接口。新增算子时，需要在IR中补充其输入输出和属性，在规划阶段确定形状、空间需求及合法参数范围，再增加相应汇编生成规则；若引入新的指令编码，还需同步扩展汇编器。由此可在保留整体流程的基础上开展软硬件协同扩展，并通过单算子与完整网络验证检查各阶段的一致性。

3.2.5 系统验证与阶段成果

本阶段已将编译器与实际硬件执行过程贯通。NPU已集成到RISC-VSoC中，并随整个SoC部署到FPGA；系统能够加载本编译器生成的YOLO网络二进制文件，由NPU执行网络计算并正确完成图像识别。这一结果表明，模型解析、参数整理、内存规划、指令编码及板端装载之间已建立可运行的完整链路，为后续在同一硬件平台上比较不同编译策略提供了实验基础。

目前已确认的是完整系统中的功能验证结果。为进一步形成可重复、可比较的实验记录，后续将补充板卡型号、工作频率、模型版本、输入尺寸及量化配置等条件，并记录NPU执行周期和端到端处理时间。现阶段尚未形成统一的帧率、功耗、加速比或数据集精度统计，相关信息按表2继续整理。

表2 FPGA验证记录与待补充实验信息

项目	当前状态	需归档的证据
系统集成与运行	已完成RISC-VSoC集成NPU及FPGA运行	系统版本、板卡照片、运行日志
YOLO图像识别	已运行编译器生成的二进制并正确识别	真实输入与输出图像、模型及样本说明
硬件与模型配置	待补充完整记录	FPGA型号、频率、模型版本、输入尺寸、量化格式
数值一致性	待补充系统性测试结果	软件参考、逐层输出、误差指标
存储与性能数据	待测量并汇总	工作区峰值、执行周期、计时边界及重复次数

图10待插入：RISC-VSoC的FPGA实物与YOLO运行结果。左侧放置FPGA开发板实物照片，标注板卡型号及图像输入、结果输出接口；右侧放置同次运行的输入图像与真实识别结果，并注明模型名称、输入尺寸和测试日期。可在下方补充指令、参数装载及执行完成的真实日志。

图10 FPGA系统运行与YOLO图像识别结果（实测材料待补充）

通过上述工作，已解决自研NPU部署过程中模型信息、数据布局和指令格式之间的转换问题，形成可重复构建的离线编译流程。后续研究将在此基础上完善逐层数值比对与性能测量，重点分析中间张量的生命周期、分支执行次序和布局约束对存储需求的影响，进一步实现执行调度与存储联合优化，并通过对照实验评价其效果。

4.已研究工作总结

4.1 Linux系统相关

按照开题报告确定的研究内容，目前已完成RISC-VSoC上的Linux系统移植及驱动开发。软件系统已形成引导程序、Linux内核、设备树和根文件系统相互配合的完整结构，并在FPGA平台完成从复位启动到用户态应用运行的整体联调。系统能够通过Linux访问SoC外设，运行基本命令和测试程序，并支持LCD显示、触摸交互及NPU控制等应用功能，完成了本阶段的Linux软件建设任务。

在启动环境方面，已实现面向目标SoC的轻量级定制引导程序，完成关键外设初始化、异常处理框架、必要SBI服务及内核启动交接。在操作系统方面，已完成与硬件能力相匹配的内核配置和功能裁剪，并基于BusyBox制作精简根文件系统。在设备支持方面，已完成设备树编写及相关外设驱动适配，将寄存器访问、中断响应和数据传输封装为Linux下的设备接口。这些工作共同解决了启动环境、内核资源描述和定制外设访问之间的衔接问题。

与开题阶段的总体方案相比，Linux部分已由各模块的设计规划落实为能够构建、部署和运行的软件系统。通过统一Makefile管理编译与镜像集成，形成了可重复使用的软件基线；通过板端启动和应用演示，验证了各部分能够协同工作。后续将在现有成果基础上整理启动日志、系统配置、镜像组成及外设运行记录，补充统一口径的体积和时间数据，并继续为AI编译器的优化实验与系统应用验证提供支持。

4.2 AI编译器相关

按照编译部分的阶段目标，目前已经完成从模型导入到NPU二进制文件生成的主要软件流程，形成模型图表示、执行计划、存储计划、参数转换和指令生成等模块，并完成在RISC-V SoC集成环境下的FPGA部署验证。编译器生成的YOLO二进制文件已经能够在该系统上运行并正确进行图像识别，表明模型到硬件执行的端到端路径已经跑通。

与开题阶段的总体方案相比，目前编译研究已结合具体NPU实现进一步明确了目标平台、输入模型和输出形式，将重点落实到静态推理的图分析、数据布局和专用指令生成。研究主线仍然是RISC-V平台上的软硬件协同与编译，后续需要在现有工程基础上继续提炼优化方法。

本课题的进一步研究应针对宏指令NPU的布局和执行约束，明确现有通用方法直接应用时遇到的问题，并通过算法设计与对照实验体现改进价值。下一阶段将优先围绕存储复用、分支调度和拼接别名的共同约束开展研究。

5.后续研究的内容和实现技术

在已完成Linux系统移植和NPU离线编译流程的基础上，后续研究将重点围绕内存规划优化与算子融合展开。前者通过分析中间张量的使用关系，减少推理过程所需的存储空间；后者结合宏指令和硬件数据通路，减少相邻运算之间的数据搬运与控制开销。两项工作均以当前可运行的YOLO网络为基线，通过单算子、典型子图和完整网络的实验验证其正确性与实际效果。

5.1 内存规划优化

当前编译器主要按照执行顺序为中间结果分配空间，虽然便于组织地址，但已经完成使用的张量仍可能保留原有区域。后续拟在现有内存规划中加入生命周期分析，记录每个张量的生产节点、全部消费者、实际物理大小和对齐要求。当某个张量已完成全部消费后，其空间可供后续张量复用，从而降低运行区的峰值需求。图11以连续执行的三个算子说明这一过程，图中仅展示复用原理，不代表具体模型的实测收益。

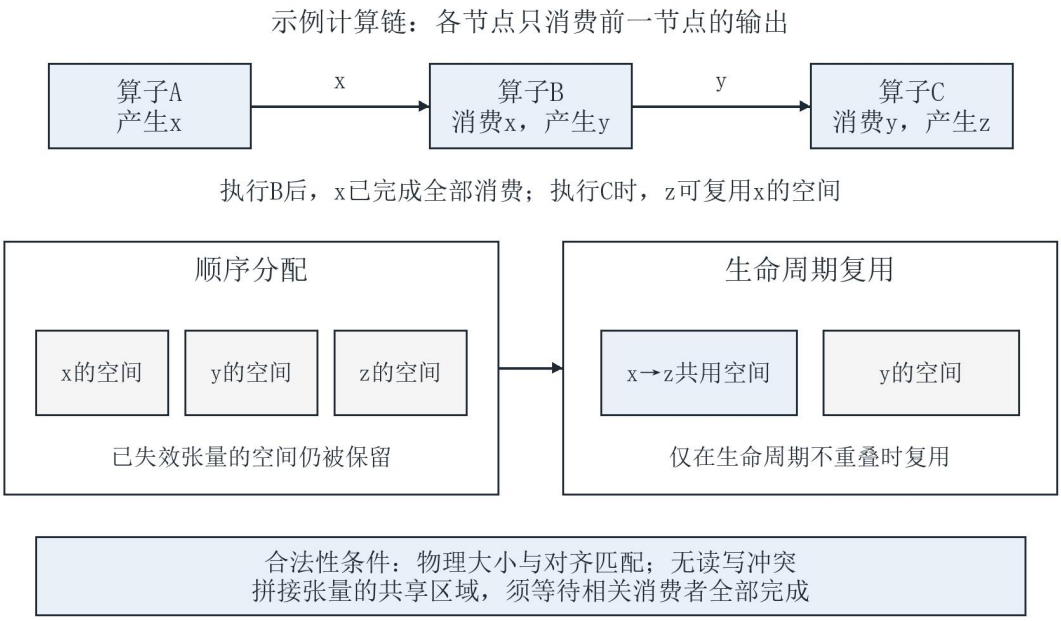


图11 基于张量生命周期的内存复用示意

实现时将以现有顺序分配作为对照，首先在固定执行顺序下建立可用空间管理与分配机制，再结合目标NPU的4通道对齐规则检查候选区域是否满足大小和访问要求。外部输入、最终输出和模型参数按各自的保留需求管理，避免在仍有用途时被回收。对已采用预布局的通道拼接，还需将底层存储区域与各分支视图关联起来，只有相关消费者均完成访问后，才允许复用共享空间。

在此基础上，将分支执行顺序与内存分配共同考虑。多分支网络在满足数据依赖的情况下可以具有不同的合法顺序，不同顺序会改变共享输入和分支输出的存活时间。MAGIS等研究说明执行调度与存储优化之间具有联合研究的空间[6]。针对本平台，拟从当前可执行节点中选择候选操作，根据新增张量大小、可释放空间以及后续拼接需求进行评分，并通过有限范围搜索选择峰值占用较低的执行方案。

该部分的研究重点是使生命周期复用、通道布局与拼接共享关系相互兼容。每个候选方案均需通过依赖顺序、地址范围和读写冲突检查，并设置固定顺序复用、仅调整分支顺序及联合优化等实验组，分析不同机制的实际贡献。评价时分别记录峰值内存、对齐填充、编译耗时和推理时间，以说明优化对部署空间与执行效率的不同影响。

5.2 算子融合

现有宏指令能够完成较大粒度的张量运算，但相邻算子分别执行时，仍可能产生中间结果写回、重新读取和重复配置的开销。后续将优先分析网络中出现频繁的

卷积与激活组合，研究在卷积输出阶段直接完成激活处理的方法。DNNFusion等工作表明，合理融合能够减少中间数据和执行开销[7]。本课题拟结合自研NPU的数据通路和指令配置开展相应扩展，其基本方式如图12所示。

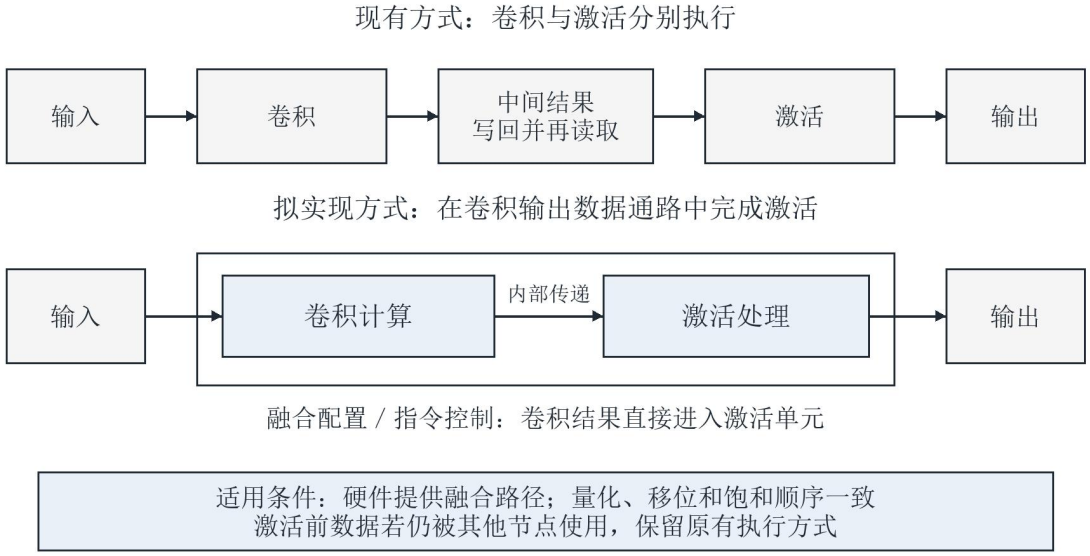


图12 卷积与激活融合的拟实现方式

具体实现将与硬件设计人员协同，明确融合所需的激活模式、输出位宽、量化移位及饱和处理顺序，并通过指令字段或配置寄存器表达相应控制信息。编译器在计算图中识别符合条件的连续算子，将其转换为融合执行描述，同步调整中间存储分配、参数组织及指令生成。对于硬件尚不支持的组合，或者激活前结果仍被其他节点使用的情况，保留原有执行方式，保证模型语义和数据依赖正确。

融合研究将以减少实际数据搬运为目标，同时考察硬件修改带来的资源和时序代价。跨层执行中的存储访问成本需要结合硬件层次分析[8]，因此将分别统计融合前后的指令数量、中间结果读写量、NPU执行周期，以及FPGA资源占用和可工作频率。在完成小规模组合验证后，再将融合路径应用于完整网络，判断其收益是否能够覆盖硬件扩展成本。

5.3 实验验证与工作安排

实验首先保证优化前后的计算正确性。针对硬件采用的整数或定点运算，建立与权重、偏置、累加、移位和饱和规则一致的软件参考；量化位置和数值格式也是验证的重要依据[9]。测试覆盖非4倍数通道、不同空间尺寸、多消费者张量及连续拼接等情况，并由单算子逐步扩展到子图和完整YOLO网络。只改变地址和执行顺

序的优化应保持同一硬件基线的输出结果；融合若改变数值处理过程，则需明确误差来源及可接受的任务指标范围。

性能实验以同一模型、输入和明确的软硬件配置开展对照，分别测量工作区峰值、指令数量、NPU执行周期，以及包含装载、预处理和后处理的端到端时间。通过重复测量记录波动范围，区分空间节省、数据搬运减少和工作频率变化所产生的影响。实验分组如表3所示，其中融合扩展组在硬件功能完成并通过正确性验证后开展。

表3 编译优化对照实验设计

实验组	配置	主要比较目的
基线组	固定顺序＋现有顺序分配	建立功能、存储与周期基线
生命周期组	固定顺序＋别名安全的空间复用	分析静态复用带来的容量收益
调度组	改变合法分支顺序，保持统一分配策略	分析调度对生命周期和峰值的影响
联合优化组	分支调度＋布局约束＋空间复用	评价联合决策相对独立阶段的收益
融合扩展组（条件性）	加入经验证的硬件后处理融合	比较访存、周期、频率与硬件资源
共同验证	相同模型、输入、频率和数值约定	逐层正确性、任务表现与可复现性

后续按“完善基线—实现优化—板端验证—论文整理”的顺序推进，优先完成生命周期复用及分支调度，再根据瓶颈分析推进算子融合。具体时间安排见表4。预期形成面向本NPU的内存规划方法、相应的融合编译支持，以及能够说明适用条件和优化效果的实验结果，并据此完成论文撰写。

表4 中期检查后的研究安排

时间	主要任务	阶段产物
2026.10—2026.11	整理硬件约束与基线；实现生命周期和别名分析	能力说明、数值测试、存储分析
2026.12—2027.01	实现布局约束下的复用及分支联合调度	算法原型、合法性检查、对照结果
2027.02—2027.03	完成多负载测试及消融；按条件评估硬件扩展	FPGA实测数据与适用范围分析
2027.03—2027.04	整理实验，完成论文写作与修改	论文初稿、修改稿与答辩材料
2027.05	完成答辩及材料归档	按学院安排完成提交

6.已发表的论文和取得的成果

无。

7.参考文献

[1] Li M, Liu Y, Liu X, et al. The Deep Learning Compiler: A Comprehensive Survey[J]. IEEE Transactions on Parallel and Distributed Systems, 2021, 32(3): 708-727. DOI: 10.1109/TPDS.2020.3030548.

[2] Chen T, Moreau T, Jiang Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[A]. In: 13th USENIX Symposium on Operating Systems Design and Implementation[C]. Berkeley: USENIX Association, 2018: 578-594. https://www.usenix.org/conference/osdi18/presentation/chen.

[3] Lattner C, Amini M, Bondhugula U, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation[A]. In: 2021 IEEE/ACM International Symposium on Code Generation and Optimization[C]. Piscataway: IEEE, 2021: 2-14. DOI: 10.1109/CGO51591.2021.9370308.

[4] Burrello A, Garofalo A, Bruschi N, et al. DORY: Automatic End-to-End Deployment of Real-World DNNs on Low-Cost IoT MCUs[J]. IEEE Transactions on Computers, 2021, 70(8): 1253-1268. DOI: 10.1109/TC.2021.3066883.

[5] Lin J, Chen W M, Lin Y, et al. MCUNet: Tiny Deep Learning on IoT Devices[A]. In: Advances in Neural Information Processing Systems 33[C]. Red Hook: Curran Associates, 2020: 11711-11722.

[6] Chen R, Ding Z, Zheng S, et al. MAGIS: Memory Optimization via Coordinated Graph Transformation and Scheduling for DNN[A]. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3[C]. New York: ACM, 2024: 607-621. DOI: 10.1145/3620666.3651330.

[7] Niu W, Guan J, Wang Y, et al. DNNFusion: Accelerating Deep Neural Networks Execution with Advanced Operator Fusion[A]. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation[C]. New York: ACM, 2021: 883-898. DOI: 10.1145/3453483.3454083.

[8] Mei L, Goetschalckx K, Symons A, et al. DeFiNES: Enabling Fast Exploration of the Depth-First Scheduling Space for DNN Accelerators through Analytical Modeling[A]. In: 2023 IEEE International Symposium on High-Performance Computer Architecture[C]. Piscataway: IEEE, 2023: 570-583. DOI: 10.1109/HPCA56546.2023.10071098.

[9] Jacob B, Kligys S, Chen B, et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference[A]. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition[C]. Piscataway: IEEE, 2018: 2704-2713. DOI: 10.1109/CVPR.2018.00286.